Terraform Landing Zone for AWS 🚀

This project sets up a modular and automated AWS landing zone using Terraform. It includes isolated environments (`pre-prod`, `prod`), reusable modules, and GitHub Actions-based CI/CD. Created as part of the Fawry internship program.

Project Structure

```
terraform-landing-zone/
├── envs/
│   ├── pre-prod/         # Environment-specific config
│   └── prod/
├── modules/              # Reusable infrastructure modules
│   ├── compute/          # EC2 instance logic
│   ├── network/          # VPC, subnets, IGW, etc.
│   └── logging/          # S3 bucket for flow logs
└── .github/workflows/    # GitHub Actions pipeline
```

What It Deploys

- **VPC** with public subnet(s)
- **EC2 Instance**
- **S3 bucket** for storing VPC Flow Logs
- **VPC Flow Logs** sent to S3
- **Remote Backend**:
- S3 for storing Terraform state
- DynamoDB for state locking

🚝Environments

Each environment (`pre-prod`, `prod`) has:

- Its own backend config (`backend.tf`)
- Its own infrastructure (`main.tf`, `outputs.tf`)

You can run each environment independently.

🚀CI/CD with GitHub Actions

CI/CD pipeline automatically runs on:

- Pushes to `pre-prod` → deploys to AWS Pre-Prod
- Pushes to `prod` → deploys to AWS Prod

Workflow steps:

- Terraform Init
- Terraform Validate
- Terraform Plan
- Terraform Apply (auto-approved)

Secrets used:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`

Set under: **GitHub → Settings → Secrets and variables → Actions**

How to Use

Prerequisites

- AWS Account (Free Tier eligible)
- Terraform CLI (`>= 1.3`)
- GitHub repo with secrets configured

🏙Setup Steps

1. Clone the repo:

```
git clone https://github.com/z1adAhmed/Fawry_terraform_task.git
cd terraform-landing-zone
```

1. [Optional] Bootstrap remote backend:

2. Create S3 bucket and DynamoDB manually

3. Or use a temporary `bootstrap.tf` file

4. Initialize environment:

```
terraform -chdir=envs/pre-prod init
```

1. Apply:

```
terraform -chdir=envs/pre-prod plan -out=tfplan
terraform -chdir=envs/pre-prod apply tfplan
```

🚚Backend Configuration

Remote backend is configured in `backend.tf`:

```
terraform {
  backend "s3" {
    bucket         = "poc-prod-vpc-logs-709efebc"
    key            = "pre-prod/terraform.tfstate"
    region         = "eu-west-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

🏫Terraform Concepts Used

- Remote state (`S3 + DynamoDB`)
- Modules for reusability
- State locking
- Workspaces & environment isolation
- GitHub Actions for CI/CD

Lessons Learned

- How to build modular Terraform code
- Secure and scalable backend management
- CI/CD integration with GitHub Actions
- Debugging IAM, state lock, and provider issues

🎪Tags

`#terraform` `#aws` `#iac` `#devops` `#fawry` `#github-actions`

License